Assignment 2: The Cryptographer's Math Toolbox

Course: CSC 4575: Applied Cryptography
Objective: Transition from bit-level symmetric ciphers to the integer algebra required for Asymmetric Cryptography (RSA/ECC).

Introduction: From Bit-Shuffling to Integer Algebra

In the first half of this course, we used AES, which operates at the bit level (XORing, S-Boxes, and bit-shifting). The security of those systems comes from **Diffusion and Confusion**.

Public-key cryptography (RSA, Diffie-Hellman, ECC) operates on very large integers. Its security depends on **One-Way Trapdoor Functions**: mathematical problems that are easy to compute in one direction but computationally "hard" to reverse without a secret key (e.g., Integer Factorization or the Discrete Logarithm Problem).

Note on Scaling: While we use small "textbook" integers (e.g., 17, 43) for clarity, Python handles **2048-bit integers** (600+ digits) using this exact same logic.

Task 1: The Euclidean Algorithm (GCD)

The Gateway Condition: Coprimality

Two integers a and n are **coprime** if $\gcd(a, n) = 1$. In RSA, your public exponent e must be coprime to the totient $\phi(n)$; if it isn't, a private key literally cannot exist.

The Mathematical Principle

The algorithm relies on the property: $\gcd(a, b) = \gcd(b, a \pmod{b})$.

Example from Narrative: $\gcd(252, 105)$

1. $252 = 2 \times 105 + 42$
2. $105 = 2 \times 42 + 21$
3. $42 = 2 \times 21 + 0$

Result: The last non-zero remainder is **21**.

[Euclidean Algorithm flowchart]

In [46]:
```python
def gcd(a, b):
    """
    Computes the greatest common divisor using the Euclidean Algorithm.

    Student Instructions:
    1. Create a while loop that runs as long as 'b' is not 0.
    2. Inside the loop, update 'a' and 'b' such that:
        - The new 'a' becomes the old 'b'.
        - The new 'b' becomes the remainder of 'a' divided by 'b'.
    3. Return 'a' after the loop finishes.

    Note for 2048-bit RSA: While we are using small integers here,
    Python's 'int' type handles numbers with hundreds of digits
    using this exact same logic.
    """
    while b != 0:
        old_a = a
        a = b
        b = old_a % b
    return a

# Test Cases
print(f"GCD(252, 105): {gcd(252, 105)}")  # Expected: 21
print(f"GCD(17, 43): {gcd(17, 43)}")     # Expected: 1 (Coprime)
try:
    assert gcd(17, 43) == 1
    print("GCD Test Passed!")
except:
    print("GCD Test Failed.")
```

```
GCD(252, 105): 21
GCD(17, 43): 1
GCD Test Passed!
```

Discussion: Euclidean Division & Modular Inverses

The Common Pitfall

In standard arithmetic, we "divide" to undo multiplication ($5x = 10 \implies x = 2$). In modular arithmetic, **division is not defined**. To undo multiplication, we must find the **Modular Multiplicative Inverse** x such that:

$$ax \equiv 1 \pmod{n}$$

Understanding Standard Euclidean Division

Before we tackle the Extended algorithm, we must master the **Division Lemma**. In cryptography, we don't care about decimals; we care about how many times a number fits into another and what is left over.

The Division Lemma

Standard division is defined as $a = q \times b + r$ where $0 \leq r < |b|$. In Python, `q = a // b` and `r = a % b`.

The "Crypto" Way of Thinking

In Python and Cryptography, we find these two values using:

1. **Floor Division (`//`):** Finds the Quotient (q).
2. **Modulo (`%`):** Finds the Remainder (r).

Example: $43 \div 17$

- How many times does 17 fit into 43? **2** times ($17 \times 2 = 34$).
- What is left over? $43 - 34 = \mathbf{9}$.
- **Equation:** $43 = 2(17) + 9$

Why this matters for the GCD

The Euclidean Algorithm works because the "commonness" between a and b is preserved in the remainder. If a number divides both 43 and 17, it *must* also divide 9. We simply repeat the division until $r = 0$.

The Goal: Finding the "Identity"

The basic algorithm finds the GCD. The **Extended** algorithm finds the integers x and y such that:

$$ax + by = \gcd(a, b)$$

These coefficients tell us how to "build" the GCD using multiples of our starting numbers. In RSA, if the GCD is 1, then x becomes our **Modular Inverse**.

The Recursive "Unwinding" Logic

To code this, we think of it in two phases:

1. **The Descent:** We keep dividing ($b\%a$) until we hit $a = 0$.
2. **The Ascent:** Once we hit the bottom, we "unwind" the math back up the chain to calculate x and y at each level.

The Update Rule: When the function returns from a deeper level of recursion with values (g, x_1, y_1), we calculate the current level's coefficients using:

- $x = y_1 - (b//a) \times x_1$
- $y = x_1$

The Mathematical Principle

While the basic Euclidean Algorithm tells us *if* a Greatest Common Divisor exists, the **Extended Euclidean Algorithm (EEA)** finds the specific coefficients x and y (known as Bézout's identity) such that:

$$ax + by = \gcd(a, b)$$

In cryptography, this is the "magic" step. If $\gcd(a, n) = 1$, then the coefficient x is the **Modular Multiplicative Inverse** of $a \pmod{n}$. Without this algorithm, we would not be able to generate RSA private keys.

Task 2: The Extended Euclidean Algorithm (EEA)

Worked Example: Find $17^{-1} \pmod{43}$

From the narrative, we track the remainders and the coefficients simultaneously using the recursive "Unwind" logic:

Step	Equation	r	x	y
Base	-	43	0	1
1	$43 = 2(17) + 9$	17	1	0
2	$17 = 1(9) + 8$	9	-2	1
3	$9 = 1(8) + 1$	1	**-5**	2

Result: $x = -5$. Since $-5 \equiv 38 \pmod{43}$, the inverse is **38**.

Implementation Strategy

The most common way to implement this is using a **recursive function**. The function should return a tuple of three values: `(gcd, x, y)`.

In [47]:
```
def extended_gcd(a, b):
    """
    Recursive implementation of EEA.
    Returns (gcd, x, y) such that ax + by = gcd.

    Instructional Walkthrough:
    1. Base Case: When 'a' reaches 0, the GCD is 'b'. At this point, we retu
    2. Recursion: Call the function with (b % a, a).
    3. The Unwind: Use the formula below to update x and y.
```

```
    """
    # --- BASE CASE ---
    if a == 0:
        # If a=0, then 0*x + b*1 = b.
        # Therefore, gcd=b, x=0, y=1.
        return b, 0, 1

    # --- RECURSIVE STEP (The Descent) ---
    # We find the GCD of the remainder and the divisor
    gcd_val, x1, y1 = extended_gcd(b % a, a)

    # --- THE UNWIND (The Ascent) ---
    x = y1 - (b // a) * x1
    y = x1

    return gcd_val, x, y

def modinv(a, m):
    g, x, y = extended_gcd(a, m)
    if g != 1: raise ValueError("No inverse exists")
    return x % m

# Verification
try:
    print(f"Inverse of 17 mod 43: {modinv(17, 43)}") # Expected: 38
except:
    print("Implement the EEA logic to calculate the inverse.")
```

```
Inverse of 17 mod 43: 38
```

Task 3: Fast Modular Exponentiation (Square-and-Multiply)

The "Big Bang" Scaling Problem

Encryption in RSA requires $c = m^e \pmod{n}$. If e is a 2048-bit number, m^e would have more digits than there are atoms in the observable universe. Calculating this directly would take longer than the time since the Big Bang.

The Binary Insight

We use **Square-and-Multiply**. For an exponent like 13 (1101_2):

$$m^{13} = m^8 \cdot m^4 \cdot m^1$$

This reduces the complexity from $O(e)$ to $O(\log e)$.

In [48]:
```
def fast_pow(base, exp, mod):
    """
    Implement Square-and-Multiply.
    Algorithm:
    1. Initialize result = 1
```

```
    2. While exp > 0:
       - If exp is odd: result = (result * base) % mod
       - base = (base * base) % mod
       - exp = exp // 2
    3. Return result
    """
    result = 1
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp = exp // 2

    return result

# Test: 5^117 mod 19
print(f"5^117 mod 19: {fast_pow(5, 117, 19)}") # Expected: 1
```

```
5^117 mod 19: 1
```

Task 4: Probabilistic Primality (Miller-Rabin)

The Need for Speed

To generate RSA keys, we need two 1024-bit primes. Deterministic tests (trial division) would take trillions of years. Instead, we use the **Miller-Rabin test**.

If a number fails, it is **definitely composite**. If it passes, it is **probably prime**. With 40 rounds of testing, the probability of error is 2^{-80}, which is statistically safer than the risk of a cosmic ray causing a hardware error in your CPU.

There are no required code edits in this task. Play with the code for understanding.

In [49]:
```
import random

def is_prime_miller_rabin(n, k=40):
    """Miller-Rabin Primality Test implementation"""
    if n <= 1: return False
    if n <= 3: return True
    if n % 2 == 0: return False

    # Write n-1 as 2^r * d
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
```

```
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False # Definitely composite
    return True # Probably prime

# test_prime = 104729 # 10,000th prime
test_prime = 84324311121765465789
print(f"Is {test_prime} prime? {is_prime_miller_rabin(test_prime)}")
```

```
Is 84324311121765465789 prime? False
```

Summary: Look Ahead to Week 6 (RSA)

Tool	Name	Role in RSA
`gcd`	Euclidean Algorithm	Verifying the Public Exponent e is valid.
`modinv`	Extended Euclidean	Calculating the Private Key d.
`fast_pow`	Square-and-Multiply	Performing Encryption (m^e) and Decryption (c^d).
`is_prime`	Miller-Rabin	Generating the massive primes p and q.